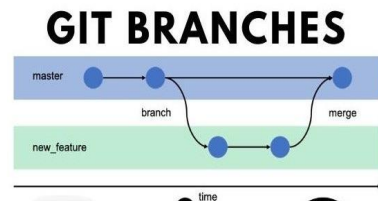
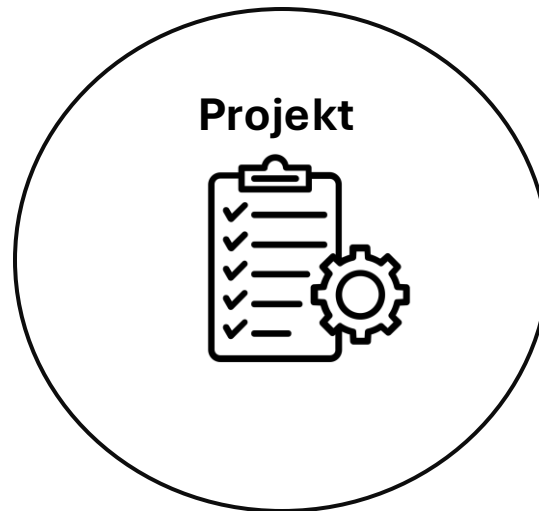


Einführung in



2025-10-06

Endprodukt heute



<https://gitlab.reutlingen-university.de>

Schritt 1: JetBrains Toolbox installieren

Windows	Mac
1. Download .exe von jetbrains.com/toolbox	1. Download .dmg von jetbrains.com/toolbox
2. Installer ausführen	2. DMG öffnen, App in Programme ziehen
3. Anweisungen folgen	3. App aus Programme-Ordner starten
4. Toolbox-Symbol in Taskleiste	4. Toolbox-Symbol in Menüleiste

Quelle: <https://www.jetbrains.com/help/toolbox-app/toolbox-app-silent-installation.html>



Schritt 2: Git installieren

Windows	Mac
1. Download von git-scm.com	1. Download von git-scm.com
2. 64-Bit Installer ausführen	2. Installer ausführen
3. Standard-Einstellungen beibehalten	3. Anweisungen folgen
4. Installation abschließen	4. Terminal: git –version zum Testen

Pfad merken (Windows): C:\Program Files\Git\bin\git.exe

Quelle: <https://git-scm.com/downloads>



Schritt 3: IntelliJ IDEA via Toolbox

1. Toolbox öffnen
2. **IntelliJ IDEA Community** suchen
3. Auf **Install** klicken (falls nach Anmeldung gefragt, mit Hochschul-Mail registrieren oder damit einloggen)
4. Warten bis fertig

Das war's.

Quelle: <https://www.jetbrains.com/help/idea/installation-guide.html>



Schritt 4: Erstes Projekt erstellen

1. IntelliJ öffnen -> New Project
2. Links: Java auswählen
3. Name: Dein Projektname (z.B. "TerminalGame")
4. Build system: Maven <- Industriestandard
5. JDK: Wird im nächsten Schritt installiert
6. Create Git repository aktivieren
7. Create klicken

Quelle: <https://www.jetbrains.com/help/idea/new-project-wizard.html>



Schritt 5: Java (JDK) installieren

IntelliJ macht das automatisch:

1. Im Projekt-Dialog Download JDK klicken
2. Vendor: Oracle OpenJDK oder Amazon Corretto
3. Version: 21 (oder neueste LTS)
4. Download klicken
5. Fertig

IntelliJ lädt Java herunter und konfiguriert es.

Quelle: <https://www.jetbrains.com/help/idea/sdk.html>

Schritt 6: Git konfigurieren in IntelliJ

Windows	Mac
File -> Settings (Strg+Alt+S)	IntelliJ IDEA -> Settings (⌘,))
Version Control -> Git	Version Control -> Git
Pfad sollte automatisch erkannt sein	Pfad sollte automatisch erkannt sein
Falls nicht: Browser -> C:\Program Files\Git\bin\git.exe	Falls nicht: /usr/bin/git
Test klicken -> Version angezeigt? Check	Test klicken -> Version angezeigt? Check

Quelle: <https://www.jetbrains.com/help/idea/set-up-a-git-repository.html>

Schritt 7: GitLab Token erstellen

Im Browser:

1. Gehe zu <https://gitlab.reutlingen-university.de>
2. Login mit Hochschul-Account
3. Oben rechts: Profilbild -> Edit Profile
4. Links: Access tokens
5. Add new token

Quelle: https://docs.gitlab.com/user/profile/personal_access_tokens/

Schritt 7: GitLab Token erstellen

Token-Einstellungen:

- Name z.B.: IntelliJ IDE Laptop
- Expiration: Semesterende
- Scopes: api, read_user, write_repository

6. Create personal access token

7. Kopieren! Wird nur einmal angezeigt

Quelle: https://docs.gitlab.com/user/profile/personal_access_tokens/

Schritt 8: In IntelliJ mit GitLab verbinden

IntelliJ:

1. File -> Settings (Windows) oder IntelliJ IDEA -> Settings (Mac)
2. Version Control -> GitLab
3. + Symbol klicken
4. Server: <https://gitlab.reutlingen-university.de>
5. Token: dein kopierter Token einfügen
6. Login in klicken
7. Erfolgreich? Super.

Quelle: <https://www.jetbrains.com/help/idea/gitlab.html>

Schritt 9: Projekt auf GitLab teilen

IntelliJ:

- Git -> GitLab -> Share project on GitLab

Dialog:

- Account: gitlab.reutlingen-university.de (ausgewählt)
- Repository name: Dein Projektname
- Visibility: Private
- Description: Optional

IntelliJ erstellt Repo, committed alle Dateien, pusht zu GitLab

Quelle: <https://www.jetbrains.com/help/idea/clone-gitlab-project.html>

Schritt 10: Projekt auf GitLab ansehen

Im Browser:

- Gehe zu <https://gitlab.reutlingen-university.de>
- Dein Projekt sollte jetzt bei deinen Projekten sichtbar sein
- Klick drauf

Du siehst:

- Alle deine Dateien
- README.md
- pom.xml (Maven-Konfiguration)

Das ist dein Remote-Repository!

Quelle: <https://www.jetbrains.com/help/idea/clone-gitlab-project.html>

Schritt 11: Teammitglieder hinzufügen

Auf GitLab (Browser):

- Öffne dein Projekt
- Links: **Manage -> Members**
- Invite members

Mitglied hinzufügen:

- Email: Hochschul-E-Mail des Teammitglieds
- **Role: Developer** (kann Code pushen)
- Invite klicken

Fertig. Teammate kann jetzt auch pushen/pullen.



Schritt 12: Programmieren, und dann?

In IntelliJ:

1. Öffne eine Datei (z.B. `Main.java`)
2. Ändere etwas (z.B. füge Code hinzu)
3. Speichern (`Strg+S` / `⌘+S`)

**Datei wird gelb markiert → Git hat Änderung erkannt.
Jetzt committen.**



Schritt 13: Commit **vs** Commit and Push

COMMIT (lokal):

- Speichert in deinem lokalen Git
- Nur auf deinem Computer
- Schnell, offline möglich
- Ermöglicht Rückkehr zu vorherigem Commit, wenn Du deine Veränderungen zurücknehmen willst

COMMIT AND PUSH (remote):

- Commit + Upload zu GitLab
- Für alle sichtbar
- Teammitglieder können pullen

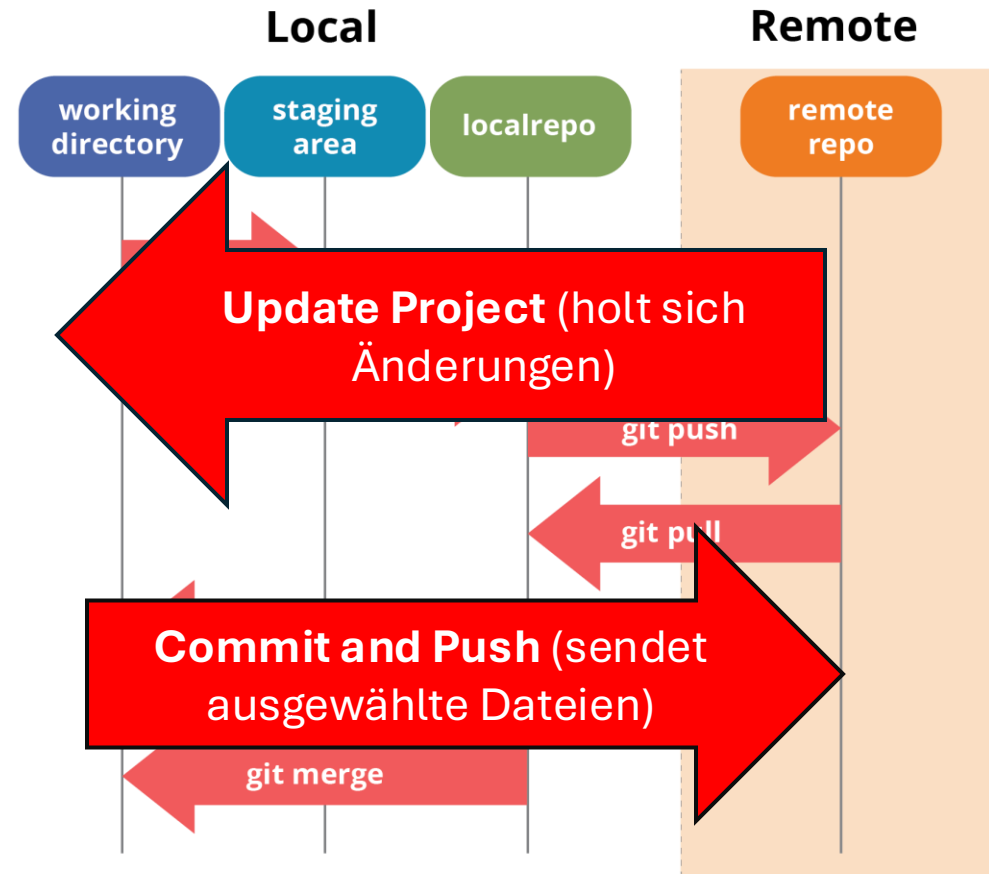
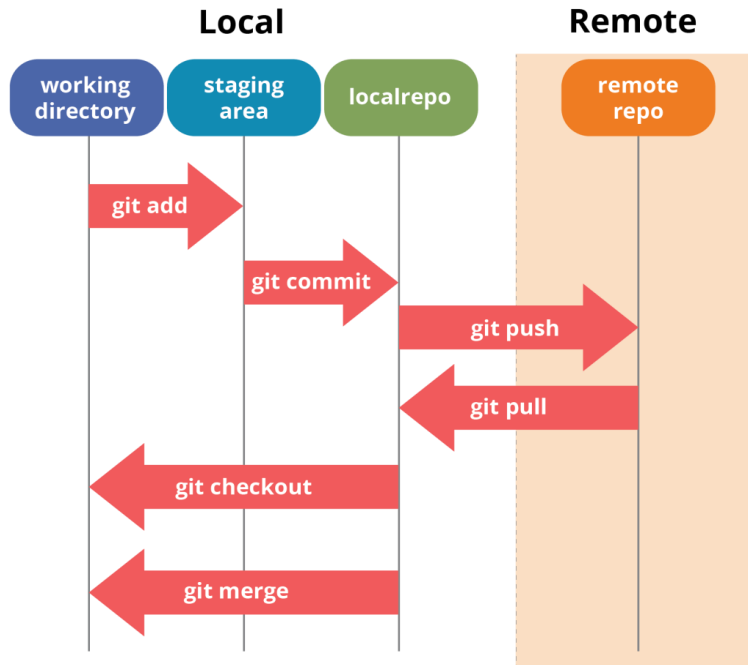
Faustregel: Immer **Commit and Push** nutzen für Team-Projekte.
Damit andere wissen, dass es Änderungen gab.

Quelle: <https://www.jetbrains.com/help/idea/commit-and-push-changes.html>



Welche GUI-Knöpfe braucht man im 2. Semester?

Das macht IntelliJ im Hintergrund:



Schritt 14: Commits vorbereiten (wir wollen ja nicht alles in einem Projekt pushen)

IntelliJ:

- **Alt+0** (Windows) oder **⌘+0** (Mac)
- Oder: Links in Toolbar auf **Commit** Icon klicken

Du siehst:

- Liste aller geänderten Dateien
- Diff-Ansicht (was wurde geändert)
- Commit Message Feld



Schritt 15: Commit and Push durchführen

Im Commit-Fenster:

- ☒ **Dateien auswählen** die du committen willst
- **Commit message** schreiben (z.B. "Add player movement")
- **Dropdown-Pfeil** neben Commit → **Commit and Push**
- Im Push-Dialog: **Push** klicken

Beim ersten Mal:

- IntelliJ fragt nach GitLab-Zugangsdaten
- **Username:** Dein Hochschul-Username
- **Password:** Dein TOKEN (nicht dein Passwort!)
- ☒ **Remember** aktivieren

Fertig. Code ist auf GitLab.

Quelle: <https://www.jetbrains.com/help/idea/commit-and-push-changes.html>



Schritt 16: Änderungen auf GitLab prüfen

Im Browser:

- GitLab-Projekt öffnen
- Links: **Code** → **Commits**
- Dein Commit sollte ganz oben stehen

Du siehst:

- ✓ Commit-Message
- ✓ Zeitstempel
- ✓ Dein Name
- ✓ Geänderte Dateien

Code ist erfolgreich gepusht. ✓

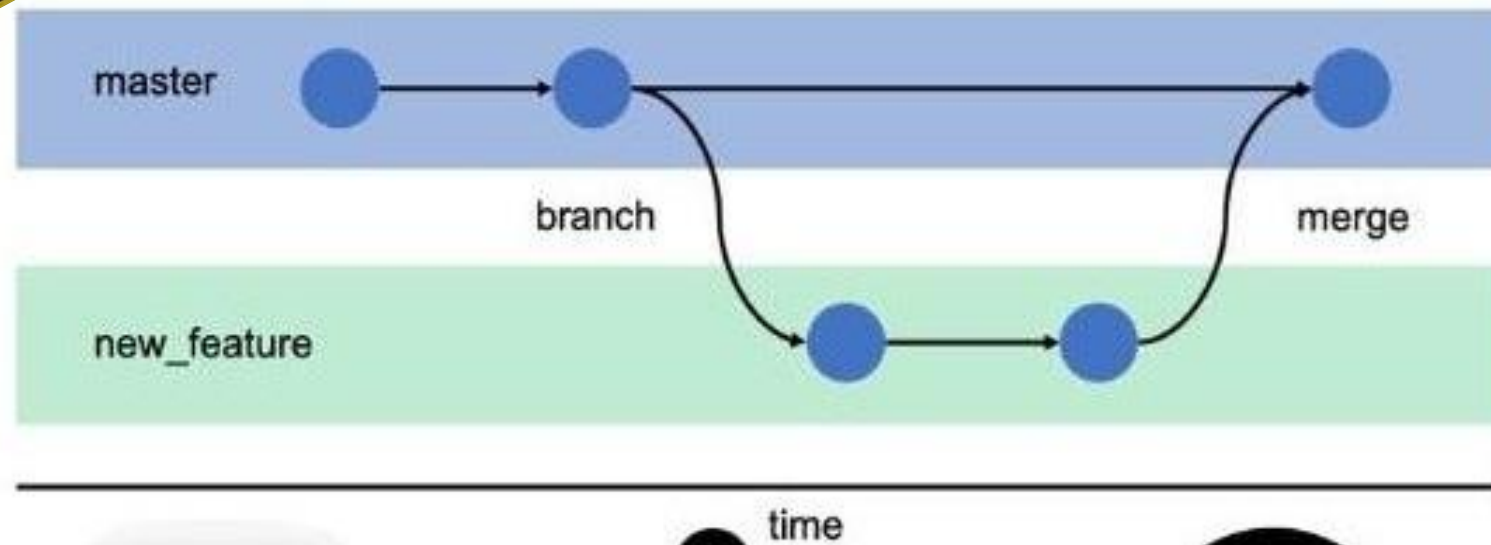
Quelle: <https://docs.gitlab.com/user/project/repository/>



Mehrere Personen im Team – totales Chaos? Was wenn gleiche Code-Abschnitte bearbeitet werden?



GIT BRANCHES



Schritt 17: Branch erstellen

IntelliJ:

- Oben rechts: **Branch-Name** in Toolbar klicken (z.B. "main")
- **New Branch** wählen

Im Dialog:

- **Name:** z.B. feature/highscore oder bugfix/player-crash
- ☒ **Checkout branch** aktiviert lassen
- **Create**

Du arbeitest jetzt auf dem neuen Branch.

Branch-Name wird in Toolbar angezeigt.



Schritt 17: Branch erstellen

IntelliJ:

- Oben rechts: **Branch-Name** in Toolbar klicken (z.B. "main")
- **New Branch** wählen

Im Dialog:

- **Name:** z.B. feature/highscore oder bugfix/player-crash
- ☒ **Checkout branch** aktiviert lassen
- **Create**

Du arbeitest jetzt auf dem neuen Branch.

Branch-Name wird in Toolbar angezeigt.



Schritt 18: Branch committen und pushen

Normale Workflow:

- Änderungen machen auf deinem Branch
- **Commit and Push** wie vorher
- Beim ersten Push: IntelliJ fragt "Branch existiert nicht remote, erstellen?"
- **Push** klicken

Branch ist jetzt auf GitLab.

Quelle: <https://www.jetbrains.com/help/idea/commit-and-push-changes.html>



Schritt 19: Branch auf GitLab ansehen

Im Browser:

- GitLab-Projekt öffnen
- Oben: **Branch-Dropdown** (steht "main")
- Klicken → Dein neuer Branch wird angezeigt
- Branch auswählen

Du siehst jetzt die Dateien auf diesem Branch.

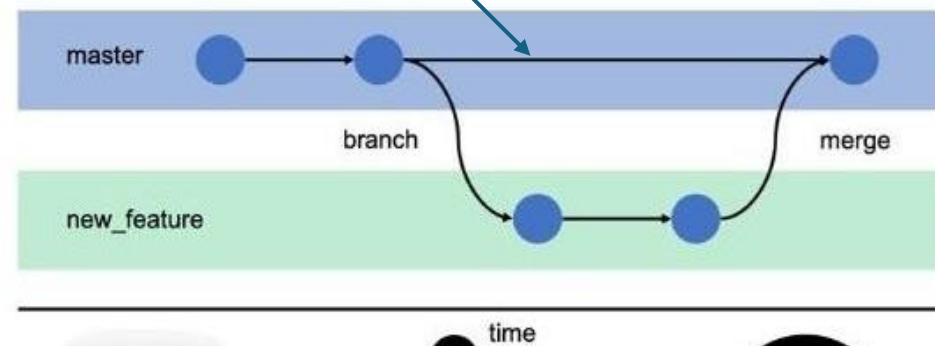
Commits auf diesem Branch: Links → Code → Commits



Mergen – Best Practice aus der Erfahrung

Was, wenn ein Kommiliton:in master verändert hat, nachdem ich bereits mit einem Branch aus master herausgegangen bin?

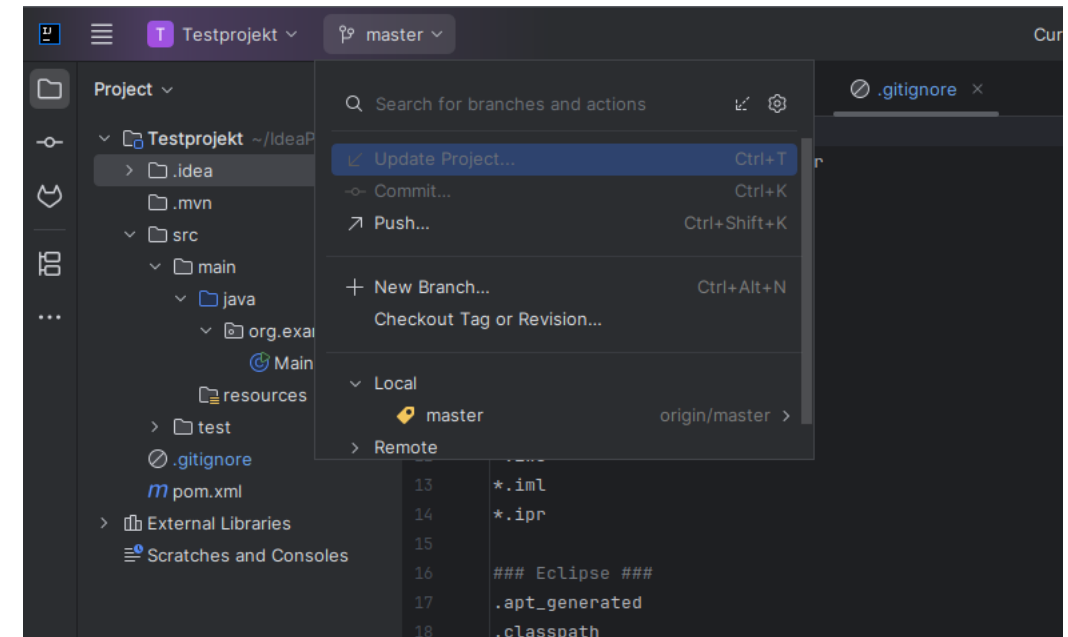
Was passiert, wenn ich meine Veränderungen in master "mergen" will?



Easy Solution:

Im Workflow immer am Start "update Project" im Git-Dropdown drücken, damit die Veränderungen im eigenen Branch und im master Branch bei einem lokal ankommen.

Wenn dann von euch bereits veränderter, aber nicht gepusheter, Code überschrieben werden soll, habt ihr eine intuitive GUI, bei der Ihr auswählen könnt, welchen Code Ihr behalten wollt.



Quelle: <https://docs.gitlab.com/user/project/repository/branches/>

Schritt 20: Merge Request im Browser erstellen

Auf GitLab:

- Projekt öffnen
- Links: **Code** → **Merge requests**
- **New merge request**
- **Source branch:** Dein Branch
- **Target branch:** main / master
- **Compare branches and continue**
- **Title + Description** ausfüllen
- **Create merge request**

Fertig. Wartet auf Review der anderen und/oder Merge klicken.

Falls es nen Fehler gibt (merge conflict), einfach in IntelliJ auf dem neusten Stand des Branches Update Project drücken.



Quelle:

https://docs.gitlab.com/user/project/merge_requests/creating_merge_requests/

Good to knows

Häufige Probleme und Fettnäpfchen

"Push rejected" → Erst **Pull** machen: **Git** → **Pull**

"Merge Conflict" → IntelliJ zeigt Merge-Dialog, wähle welche Änderungen du behalten willst

"Authentication failed" → Token abgelaufen: Neuen erstellen auf GitLab

"Can't find Git" → Settings → Version Control → Git → Pfad manuell setzen



Good to knows

Häufige Probleme und Fettnäpfchen

Nur pushen, was wirklich geteilt werden soll.

-> keine dateien aus:

.idea

node_modules

.git

cmake

Allgemein: keine build dateien oder configurations, die spezifisch für deine Maschine sind.



Good to knows

Häufige Probleme und Fettnäpfchen



Wenn alles kaputt ist, wenn GIT gar nicht mehr geht, cloned neu und kopiert eure Änderungen in den neuen Ordner.

New... -> Project from Version Control ... -> GitLab

